



**MATLAB GUIDE
for Feedback
Control Instrumentation**

Feedback Instruments Ltd., Park Road, Crowborough, East Sussex, TN6 2QX, UK
Telephone: +44 (0) 1892 653322, Fax: +44 (0) 1892 663719
E-mail: feedback@feedback-instruments.com, Website: www.feedback-instruments.com

Manual: 33-000-V73 Ed03 082013
Feedback Part No. 1160-33000V73

THE HEALTH AND SAFETY AT WORK ACT 1974

We are required under the Health and Safety at Work Act 1974, to make available to users of this equipment certain information regarding its safe use.

The equipment, when used in normal or prescribed applications within the parameters set for its mechanical and electrical performance, should not cause any danger or hazard to health or safety if normal engineering practices are observed and they are used in accordance with the instructions supplied.

If, in specific cases, circumstances exist in which a potential hazard may be brought about by careless or improper use, these will be pointed out and the necessary precautions emphasised.

While we provide the fullest possible user information relating to the proper use of this equipment, if there is any doubt whatsoever about any aspect, the user should contact the Product Safety Officer at Feedback Instruments Limited, Crowborough.

This equipment should not be used by inexperienced users unless they are under supervision.

We are required by European Directives to indicate on our equipment panels certain areas and warnings that require attention by the user. These have been indicated in the specified way by yellow labels with black printing, the meaning of any labels that may be fixed to the instrument are shown below:



CAUTION -
RISK OF
DANGER



CAUTION -
RISK OF
ELECTRIC SHOCK



CAUTION -
ELECTROSTATIC
SENSITIVE DEVICE

Refer to accompanying documents

PRODUCT IMPROVEMENTS

We maintain a policy of continuous product improvement by incorporating the latest developments and components into our equipment, even up to the time of dispatch.

All major changes are incorporated into up-dated editions of our manuals and this manual was believed to be correct at the time of printing. However, some product changes which do not affect the instructional capability of the equipment, may not be included until it is necessary to incorporate other significant changes.

COMPONENT REPLACEMENT

Where components are of a 'Safety Critical' nature, i.e. all components involved with the supply or carrying of voltages at supply potential or higher, these must be replaced with components of equal international safety approval in order to maintain full equipment safety.

In order to maintain compliance with international directives, all replacement components should be identical to those originally supplied.

Any component may be ordered direct from Feedback or its agents by quoting the following information:

- | | |
|------------------------|----------------------------|
| 1. Equipment type | 3. Component value |
| 2. Component reference | 4. Equipment serial number |

Components can often be replaced by alternatives available locally, however we cannot therefore guarantee continued performance either to published specification or compliance with international standards.

CE DECLARATION CONCERNING ELECTROMAGNETIC COMPATIBILITY

Should this equipment be used outside the classroom, laboratory study area or similar such place for which it is designed and sold then Feedback Instruments Ltd hereby states that conformity with the protection requirements of the European Community Electromagnetic Compatibility Directive (89/336/EEC) may be invalidated and could lead to prosecution.

This equipment, when operated in accordance with the supplied documentation, does not cause electromagnetic disturbance outside its immediate electromagnetic environment.

COPYRIGHT NOTICE

© **Feedback Instruments Limited**

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without the prior permission of Feedback Instruments Limited.

ACKNOWLEDGEMENTS

Feedback Instruments Ltd acknowledge all trademarks.

MATLAB is a registered trademark of Mathworks Inc.

Table of Contents

Manual overview	1
MATLAB® introduction.....	2
MATLAB® Desktop and basics	4
Exercise 1 – MATLAB® environment basics.....	5
Exercise 2 – Creating and executing an m-file	8
Simulink® introduction.....	11
Simulink basics.....	11
Exercise 3 – Creating and simulating a Simulink model.....	13
MATLAB® guide for Feedback exercises.....	16
Phenomenological model testing.....	16
Exercise 4 – Phenomenological models testing.....	16
Model Identification	22
Exercise 5 – Model identification	22
PID controller design	30
Exercise 6 – PID controller design	30

Manual overview

The manual serves as a guide to MATLAB® and Simulink environment specifically explaining the functionality and functions needed to carry out control experiments using the Feedback Control Instrumentation.

In the beginning some general remarks concerning MATLAB® and Simulink are given. Later sections provide support for the exercises described in the Control Experiments manuals of the following control devices: Precision Modular Servo, Pendulum, Twin Rotor MIMO System, Coupled Tanks, Maglev and Digital Servo.

The manual includes exercises so the user can get in touch with the basic Matlab and Simulink functions.

MATLAB® introduction

The high level technical computing language MATLAB® serves as an environment for algorithm development, data analysis and visualisation to name but a few. Together with Simulink and additional toolboxes it allows control system design and analysis, which can be later implemented in real time applications with the use of Real Time Workshop. Together with an Advantech PCI card such a setup constitutes an impressive digital control system development environment.

Feedback Instruments control applications are designed to be compatible with such environment. Together with already prepared applications documentation is supplied to guide the user through the design process in MATLAB®. The following crucial development steps are described:

- Phenomenological process models
- Dynamics analysis
- Discrete models identification
- Controller design
- Controller tests on the model
- Controller implementation in real time applications
- Various control strategies implementation
- Data visualisation

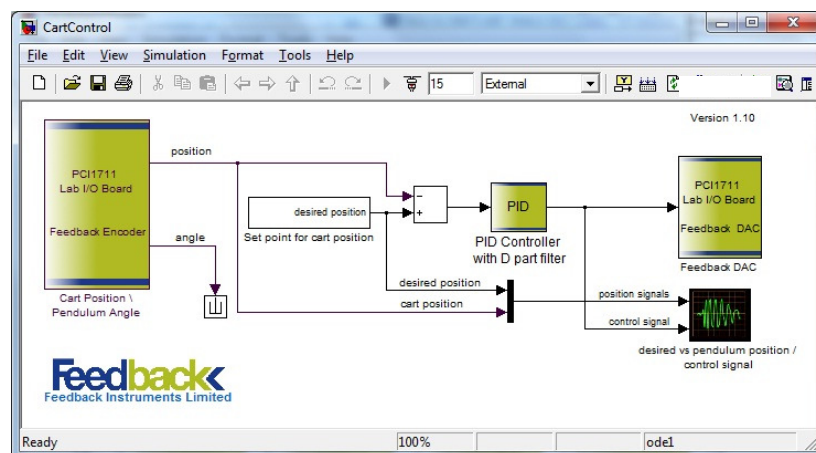


Figure 1 Real-time model

The **phenomenological process models** or the so called first principle models are designed in Simulink to provide first models for the user to test. Model linearisation is discussed and the ways of simple dynamics analysis like bode diagrams poles and zeros maps are introduced.

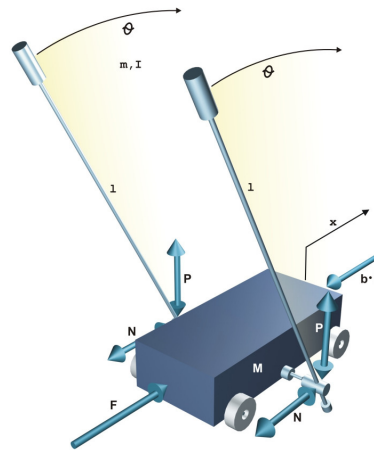


Figure 2 Phenomenological model description example

To obtain accurate models **identification procedures** incorporating MATLAB® functions are described. The user has a chance to go step by step through the discrete models identification.

One of the obtained models is used for the **controllers design**. PID control is explained and a guide is given for PID controller design, testing, tuning and implementation on the model. Root locus technique is used to picture the changes that PID controller tuning inflicts on the control system performance.

The designed controllers are prepared in Simulink to be built and tested through the **Real Time Workop** on Feedback Control Instrumentation like Pendulum, Precision Modular Servo, Twin Rotor MIMO System, Coupled Tanks, Magnetic Levitation, Digital Servo.

MATLAB® Desktop and basics

By default the MATLAB® desktop will open as presented in Figure 3.

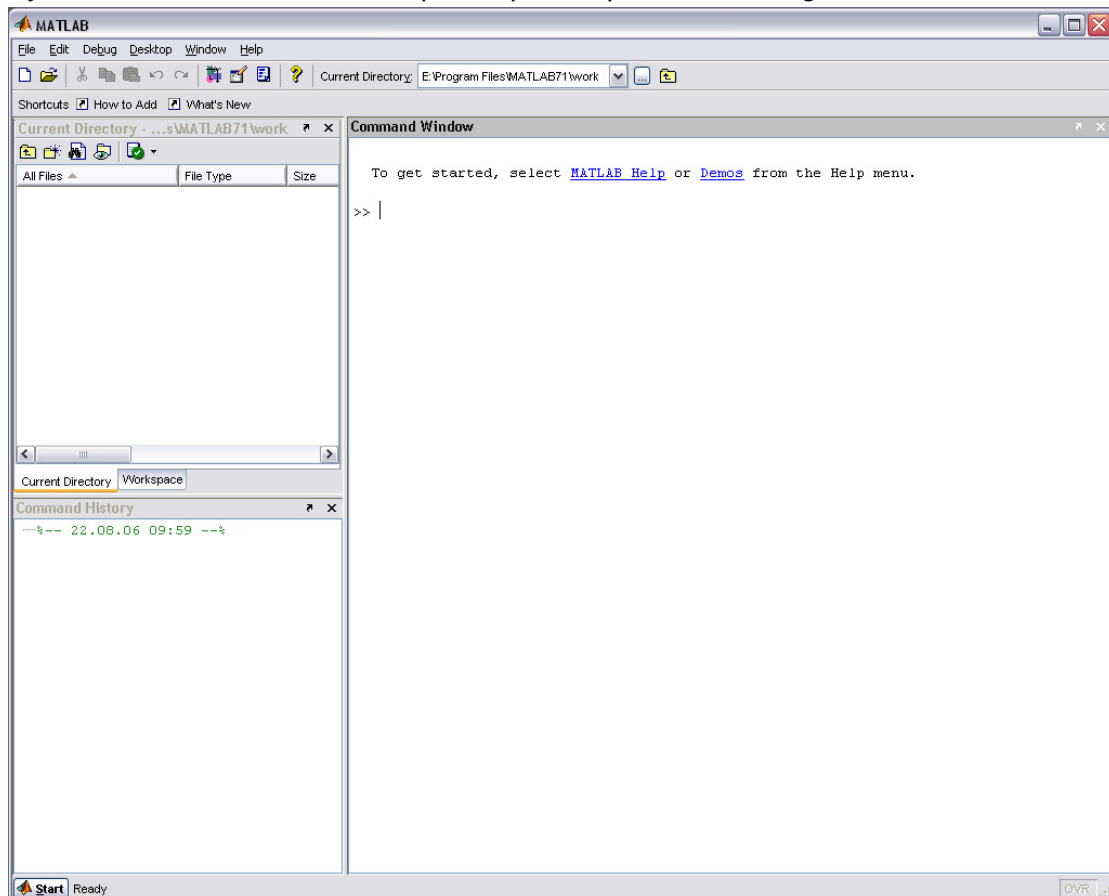


Figure 3 MATLAB® Desktop

You may arrange the layout according to preferences. The most important window is the **Command Window** marked in Figure 3. All of the commands are input there and can be executed by typing them and pressing 'Return / Enter'. You may use it also for directories change, workspace declared variable lookup, generating help on functions and toolboxes etc.

All of the commands are stored in **Command History** (Figure 3). They can be dragged from there to the Command Window and executed. You can also double click on the command to execute it immediately.

The **Current Directory** and **Workspace** window will help you organize the work.

The first exercise will let get you acquainted with the basic MATLAB® environment features.

Exercise 1 – MATLAB® environment basics

Introduction

Follow the instruction given below to get acquainted with the basic MATLAB features.

Walkthrough

- Open MATLAB and click on the 'Browse for folder' (Figure 3) button. Select the folder you want to work with.
- Type 'help' in the command window and press 'Return / Enter'. The Help topics will be displayed in the command window.
- Click on the help topic you want to view. You may also view the help topic by typing:

```
>> help control
```

in the command window – to view Control Toolbox help.

This will list all the available functions within the toolbox. To get more insight into functions you may type

```
>> help <function name>
```

for example:

```
>> help tf
```

Help on transfer function creating with the use of *tf* will be displayed. The function help information is structured in the following way:

Description of what the function does
Syntax possibilities i.e. <i>sys = tf(num,den)</i> , <i>sys = tf(num,den,ts)</i> ...
Data format information
Examples
Functions worth seeing – relating to the above function
Help file link

The complete help structure is not always available.

- Create a transfer function by typing:

```
>> sys = tf( [1] , [1 2] )
```

'sys' will be now declared in Workspace. Use the 'bode' command to view its Bode plot:

```
>> bode(sys)
```

Execute:

```
>> whos
```

to view the Workspace declared variables. Use:

```
>> clear <variable name>
```

for example:

```
>> clear sys
```

to delete a specific Workspace variable or

```
>> clear all
```

to clear all Workspace variables.

- The variables present in Workspace can be saved into a file (*filename.mat*). Use the following command to save the variables from workspace:

```
>> save <file name> <variables list>
```

for example:

```
>> save t_function sys
```

which will save a variable named 'sys' into a file '*t_function.mat*'. The variable can be loaded into workspace using the following command:

```
>> load <file name>
```

for example:

```
>> load t_function
```

All of the variables stored in such file will be loaded into workspace without prompt for overwriting.

Tip. If the command you want to use again has been executed in the same session you may type its first letter or letters and use the 'UP ARROW' to recall all of the commands beginning with such letters.

Create a transfer function using the '*tf*' command:

```
>> sys = tf( [1] , [1 2] )
```

type 's' in the command window:

```
>> s
```

use the UP ARROW to complete the line with the commands stored in the today's command history. The line should be completed to yield:

```
>> sys = tf( [1] , [1 2] )
```

Exercise 2 – Creating and executing an m-file

Introduction

The exercise will help you create, save and use m-files to repeatedly execute lists of commands.

Walkthrough

- To open a new m-file or to edit an existing one you may type:

```
>> edit <file name>
```

for example

```
>> edit signal_plot
```

an m-file editor will open (Figure 4).

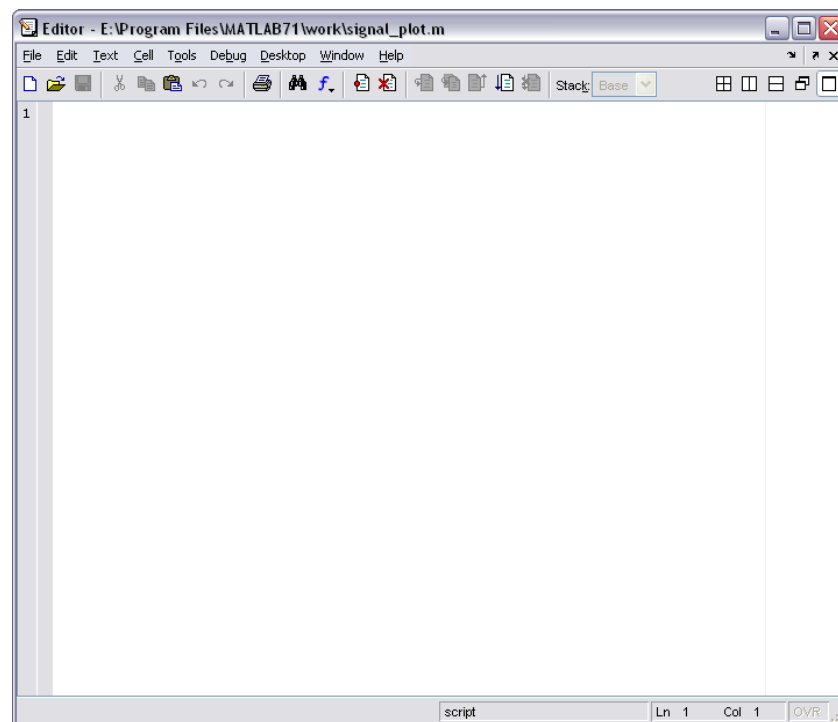


Figure 4 m-file editor window

- Usually the m-file starts with a few commented lines, which are displayed when the following command is executed:

```
>> help <m-file name>
```

Start editing the m-file by typing the following lines:

```
% A sinusoidal signal plot  
% created on dd.mm.yy
```

Save the m-file from time to time to avoid data loss.

- The following lines constitute an m-file plotting a sinusoidal signal:

```
%creating a time vector  
t = 0:0.01:2;  
  
%sine frequency  
f = 2;  
  
%sine signal  
sig = sin(2*pi*f*t);  
  
%plotting the signal  
figure  
plot(t,sig,'r-')  
xlabel('t [s]')  
ylabel('value')  
title('Sine signal')  
legend('Sine wave')
```

The ';' at the end of the line is only needed when you do not want the created variable to be displayed when the m-file is executed. This is especially useful when the program incorporates loops.

To display the created variable value during programme execution do not use the ';' at the end of line.

- To execute the m-file save it and press F5 or return to the command window. Type its name and run by pressing 'Return / Enter'.

```
>> signal_plot
```

The following figure will open after execution (Figure 5).

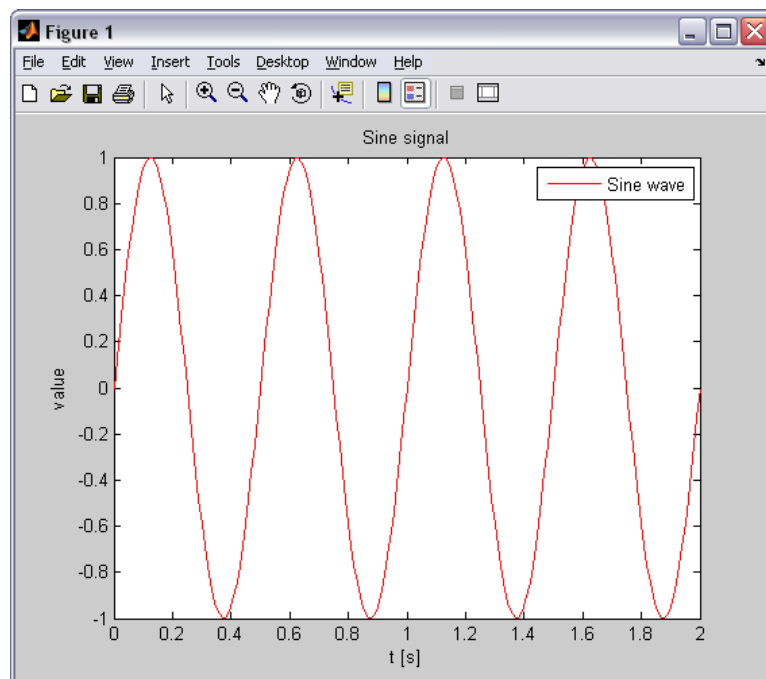


Figure 5 Resulting figure

To learn more about the functions that have been used in the example use the 'help' command.

To learn more about the possibilities of MATLAB functions use, which are not described in this manual refer to the MathWorks web site:

<http://www.mathworks.com>

or use MATLAB help.

Simulink® introduction

Simulink® is an extensive tool for simulation and together with Real Time Workshop an environment for Real Time plant control. Simulink® helps you to model, organise and test a process. A vast number and variety of already prepared blocks representing different operations are available: continuous, discrete, math operations, logic, discontinuous, signal operations, sources, sinks etc. The user can interconnect the blocks in one model and then analyse the behaviour through simulation.

Simulink basics

There are two ways to open Simulink, either by using the MATLAB Simulink icon



, or by executing the following command in the MATLAB command window:

```
>> simulink
```

The following Simulink Library Browser window should open (Figure 6):

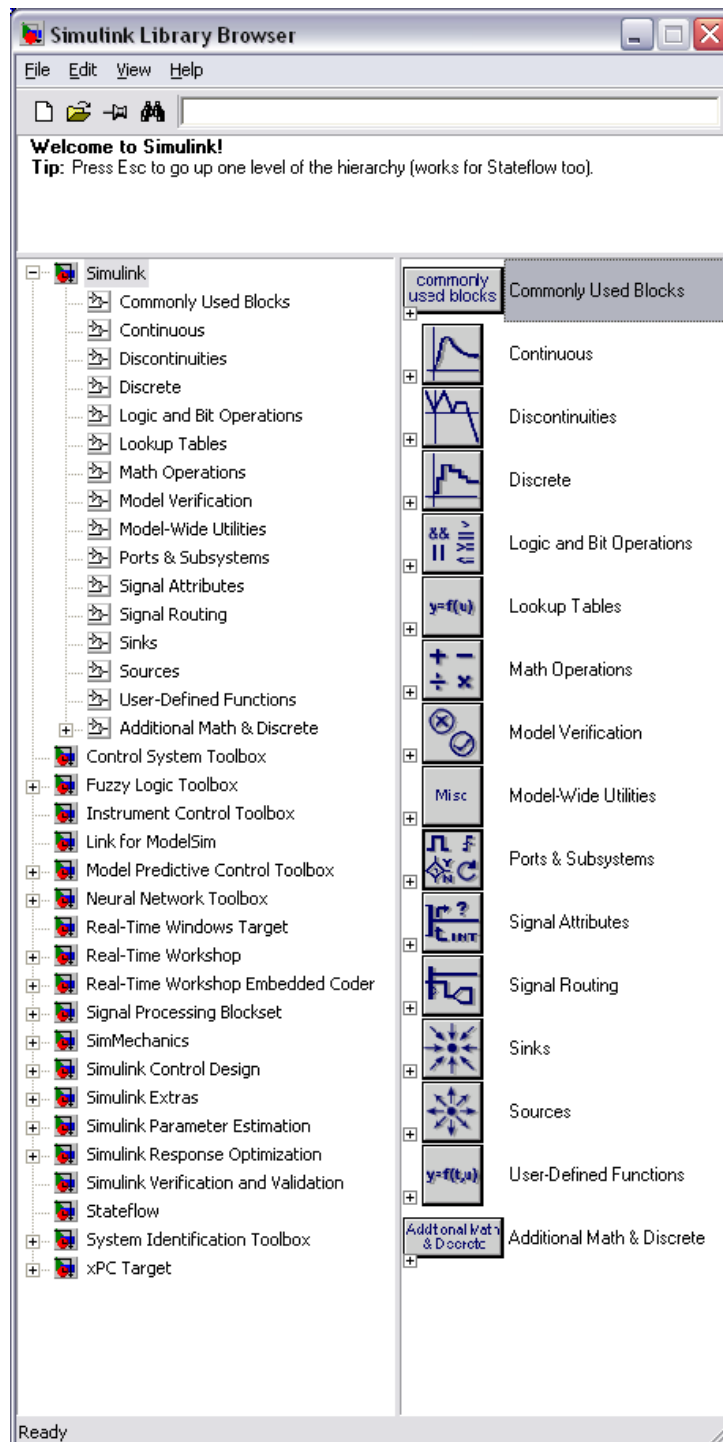


Figure 6 Simulink Library Browser window

This window may be used for new model creating, model opening and model library browsing. The following exercise will guide you through a simple model creating and running process.

Exercise 3 – Creating and simulating a Simulink model

Introduction

The exercise will help you create, save and use Simulink models for simulation.

Walkthrough

- You may open a new Simulink model by pressing the '*Create a new model*' icon . The following model window will open (Figure 7):

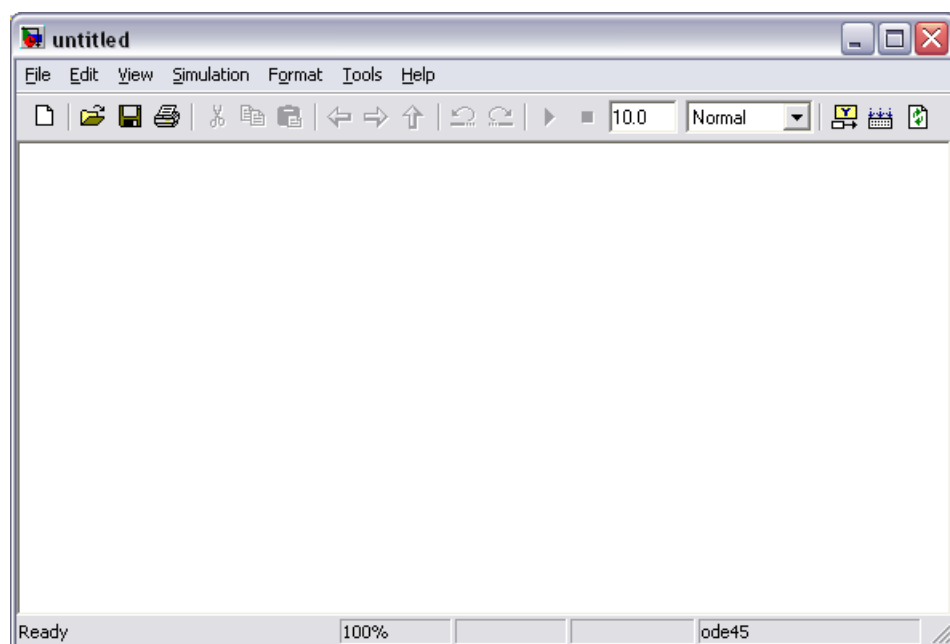


Figure 7 Model Simulink window

- Return to the Library Browser window and click on the '*Continuous*' models category. All of the blocks available under that category should appear in the right frame of the library window.
- Select the '*Transfer Fcn*' block and drag it over to the newly opened Simulink model.

- Using the same technique place the 'scope' from the 'Sinks' category and a 'Step' from the 'Sources' category. Interconnect the output of the transfer function with the scope input and the step output with the transfer function input. The obtained model should be as the one presented in (Figure 8)

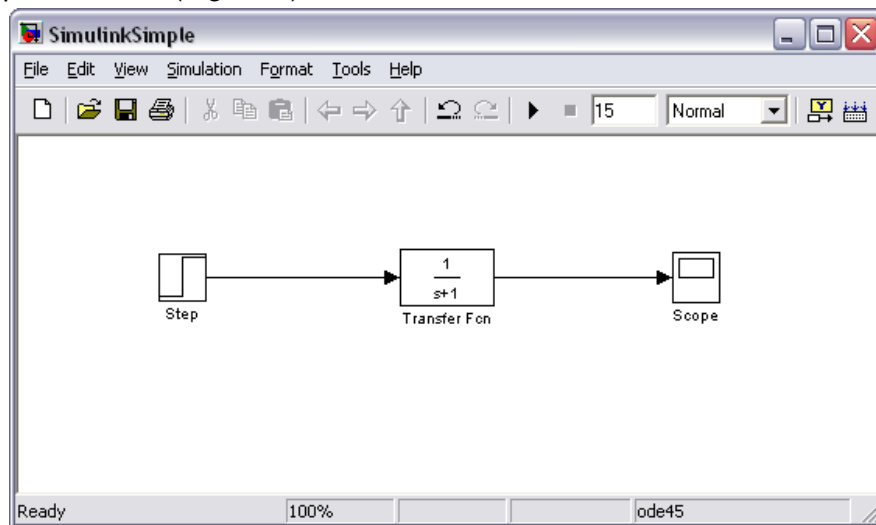


Figure 8 Simple Simulink model

- Save the model by using the save  icon.
- You may vary the simulation time by changing the value in the 'Simulation stop time' edit box (Figure 8). Set the stop time to value of 15 [s].
- Open the scope by double clicking on it and run the simulation using the 'Start simulation'  button (Figure 8). The scope should display the following result (Figure 9 – use the binocular  button to autoscale):

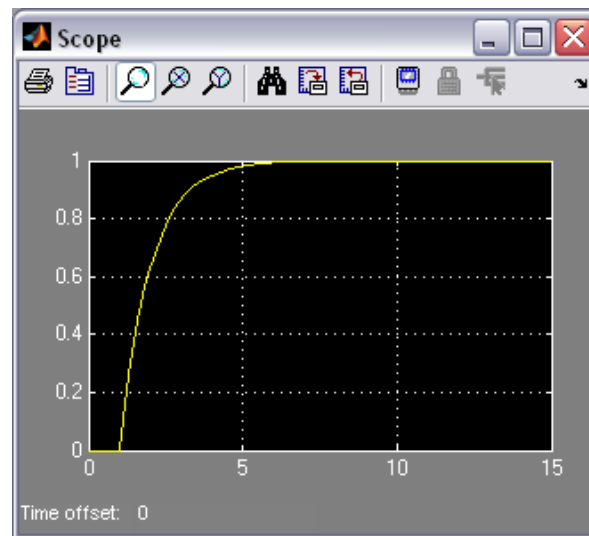


Figure 9 Scope result

- The result you have obtained is the step response of the model described by the Transfer Fcn block. You may experiment with the parameters of all of the blocks. To open parameters menus double click on the block.
- For more information about Simulink refer to MathWorks web site <http://www.mathworks.com> or use the Simulink Help option in the Simulink Library Browser menu.

MATLAB[®] guide for Feedback exercises

Previous section was a brief introduction to the MATLAB[®] and Simulink environment. If further assistance is required please refer to the MathWorks manuals and help files to obtain more information about the functionality.

The following sections will give guidance to the users carrying out control experiments using the Feedback control instrumentation: Pendulum, Modular Servo, Twin Rotor MIMO System, Coupled Tanks, Maglev and Digital Servo.

Phenomenological model testing

Each control instrument is supplied with a phenomenological model. The same model is designed in Simulink. If you look inside each of the models (Double click on the model or in case of masking right-click on the model and choose '*Look under mask*' to view subsystems) you will see that the blocks they are built of are nothing else but block representations of the differential equations describing the dynamics of the instrument. Using the instructions given in the previous section and given a specific set of differential equations you should be able to create a similar phenomenological model on your own using Simulink.

Since the models are delivered with every control device at this point it is necessary to become acquainted with the methods of models testing and data handling.

Exercise 4 – Phenomenological models testing

Introduction

The exercise will help you run the Simulation models delivered with the equipment, change the simulation parameters, observe data plots with scopes, plot the results in figures and export data to files.

Walkthrough

- Using the Matlab Desktop open the *PhenomExample.mdl* file. After device software installation it should be located in the following folder:

your_path\Feedback\MatlabGuide

The following window should open (Figure 10):

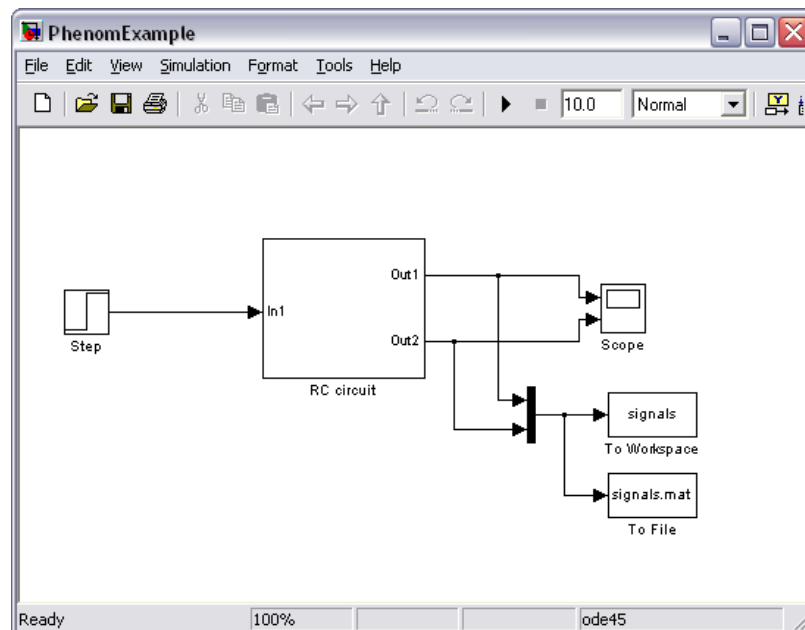


Figure 10 Phenomenological model example

- To view the RC circuit subsystem blocks right-click on the system and select the '*Look under mask*' option. The block equivalent of the following RC circuit equation should appear:

$$R \cdot i(t) + \frac{1}{C} \int_0^{t_1} i(t) dt = u_z(t), \quad (1)$$

which in Laplace domain assuming 0 initial conditions is equivalent to:

$$R \cdot I(s) + \frac{I(s)}{Cs} = U_z(s), \quad (2)$$

$$I(s) \cdot \left(R + \frac{1}{Cs}\right) = U_z(s), \quad (3)$$

$$I(s) = \frac{1}{\left(R + \frac{1}{Cs}\right)} U_z(s), \quad (4)$$

$$I(s) = \frac{Cs}{(RCs + 1)} U_z(s), \quad (5)$$

Since the capacitor voltage is given by:

$$u_c = \frac{1}{C} \int_0^{t_1} i(t) dt \rightarrow U_c = \frac{I(s)}{Cs}, \quad (6)$$

the capacitor voltage will depend on the input voltage in the following way:

$$U_c(s) = \frac{I(s)}{Cs} = \frac{Cs}{(RCs + 1) \cdot Cs} \cdot U_z(s) = \frac{1}{(RCs + 1)} \cdot U_z(s), \quad (7)$$

In such form the model can be created with the use of Transfer Fcn block (Figure 11) or based on equation (7) the model output can be estimated as:

$$U_c(s) = \frac{1}{(RCs + 1)} \cdot U_z(s), \quad (8)$$

$$RCsU_c(s) + U_c(s) = U_z(s), \quad (9)$$

which in time domain corresponds to

$$RC\dot{u}_c(t) + u_c(t) = u_z(t), \quad (10)$$

$$\dot{u}_c(t) = \frac{u_z(t) - u_c(t)}{RC}, \quad (11)$$

$$u_c(t) = \int_0^{t_1} \frac{u_z(t) - u_c(t)}{RC} dt. \quad (12)$$

The integral block can be used to calculate the output of the system (Figure 11).

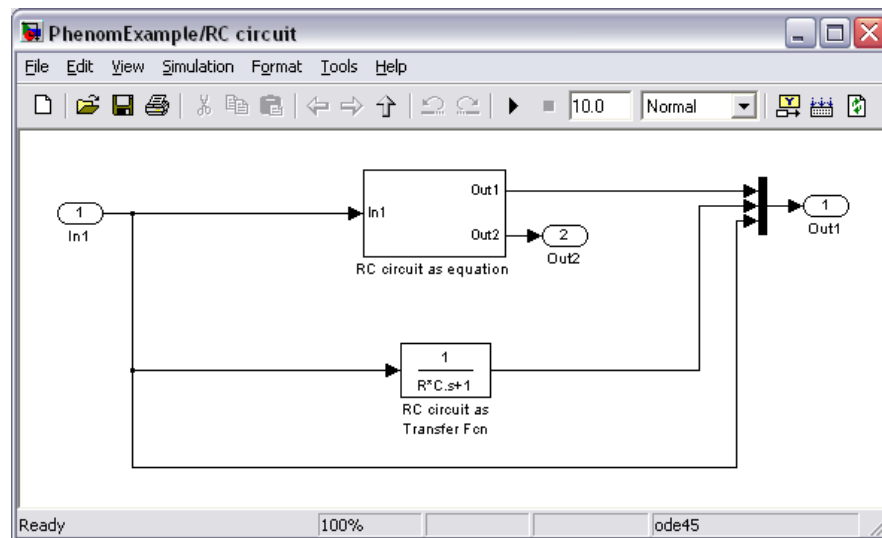


Figure 11 Phenomenological model in two forms

- Run the simulation using the 'Start simulation' button (Figure 10). Open the scope to view the simulation signals. To open the scope parameters window click on the icon. To set axes properties right-click on the scope plot and select 'Axes properties'.
- The signals can be saved to Workspace with the 'To Workspace' block. Double click on the block to open the properties window. The following options can be changed:

Variable name – the name under which the variable is stored in Workspace

Limit data points to last – point limit, inf – no limit

Decimation – every which sample to store

Sample time – signals sample time

Save format – often set to 'Array' for data handling convenience

Similar options are available in the 'To File' block. This blocks allows for data saving directly in a mat-file.

- With the data stored in Workspace or file it can be plotted in a figure, which is a convenient tool for data plotting and plots exporting to files like .jpg, .eps and many more.

After running the simulation you will have a '*signals*' variable available in Workspace, which you may check by executing the following command in the command window:

```
>> whos
```

The list of all variables should appear.

To plot the signals you can use the plot command. The results are stored in Array in separate columns form thus to obtain the plot of a specific variable (signal) you must address the appropriate row of the '*signals*' array:

```
>> plot(signals(:,1))
```

That command will plot the signal from the first column of the 'signals' array. The ':' symbol denotes that all of the rows are referenced. The following plot should appear (Figure 12).

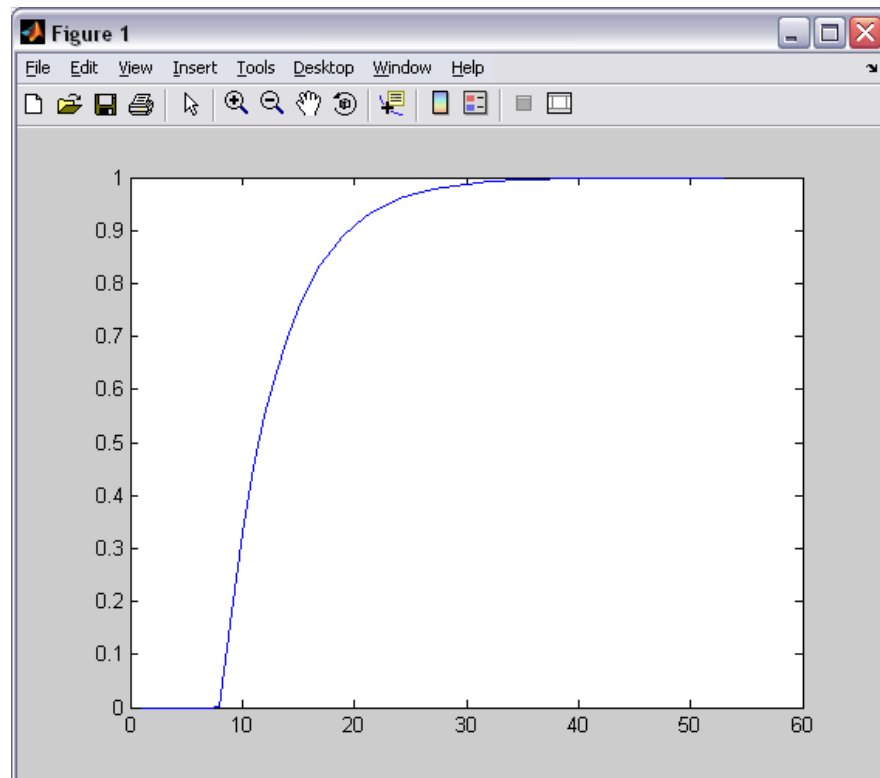


Figure 12 Simulation signal plot

Use the following command to plot the data against the simulation time vector:

```
>> plot(tout,signals(:,1))
```

You may also use plot function options to change the colour, style etc. Use the 'legend', 'xlabel', 'ylabel', 'title' command to place description in the figure. Refer to the 'MATLAB® introduction' section for more information or use MathWorks help.

To save and export figures to a file use the *File->Save* and *File->->Save As* menu options

Model Identification

In every Feedback exercise a discrete linear model is identified. The theoretical grounds for discrete model identification are given in each of the manuals. This section however presents how the MATLAB identification toolbox functions can be used to aid the identification process.

Each of the identification Real-time Simulink models is constructed in the same way although the parameters may differ. Identification applications refer to the external equipment and are run in Real-time.

Identification applications may have different sampling time. Refer to the Control Experiment manual on that exercise to check the sampling time. This information will be needed during the identification experiment.

To carry out the identification experiment on your device with the use of Matlab Identification Toolbox follow the steps presented below in the identification exercise.

Exercise 5 – Model identification

Introduction

This exercise serves as a guide for all of the Real-time identification tasks in the Feedback Control Experiments manuals for Feedback Control Instrumentation.

Walkthrough

- Open the appropriate (dedicated to the control instrument – *VelocityModel_Ident.mdl* for Modular Servo) Simulink Real-Time model to perform an identification experiment. Prepare the device to be identified following proper precautions. Make sure the device power is OFF.
- The excitation signals are already prepared. You may need to reduce the excitation signal amplitude for some devices if such an opportunity has been provided. As an example the Modular Servo model identification model is presented (Figure 13)

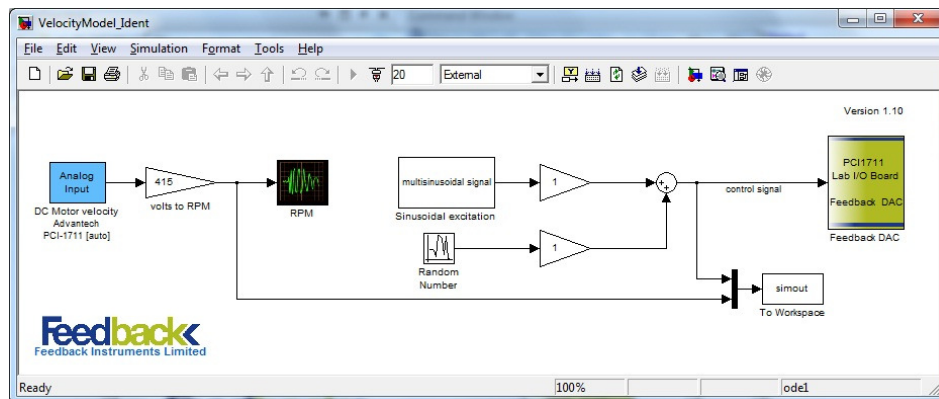


Figure 13 Modular Servo identification Simulink model

- Compile the model by pressing 'CTRL+B'. Wait for the compilation to finish. The "Successful completion of Real-Time Workshop build procedure for model: <Model Name>" statement should appear in the command window when the compilation has finished.

Press the 'Connect to target'  button. You may turn the device power ON.

Press the 'Start simulation'  button. The identification experiment will be carried out.

- When the experiment finishes due to the fact that the 'To Workspace' block has been used some of the necessary signals will be available in Workspace. Use the following command to assign the output signals to the proper variables:

```
>> u=signal(:,1)
>> y=signal(:,2)
```

Important. In Workspace the 'signal' array will be available. Be careful when you assign the proper array columns to the u and y variables. The command `signal(:,1)` means that a variable stored in the first column is assigned. The column in which the variable is stored in the 'To Workspace' block array depends on the input number of the multiplexer placed in front of that block. Thus variable connected to the 1st multiplexer input will be stored in the 1st column etc.

- Use the `detrend()` function to remove all the linear trends from the data vectors u and y .

- Type *ident* at the Matlab command line. The identification interface will open (Figure 14).

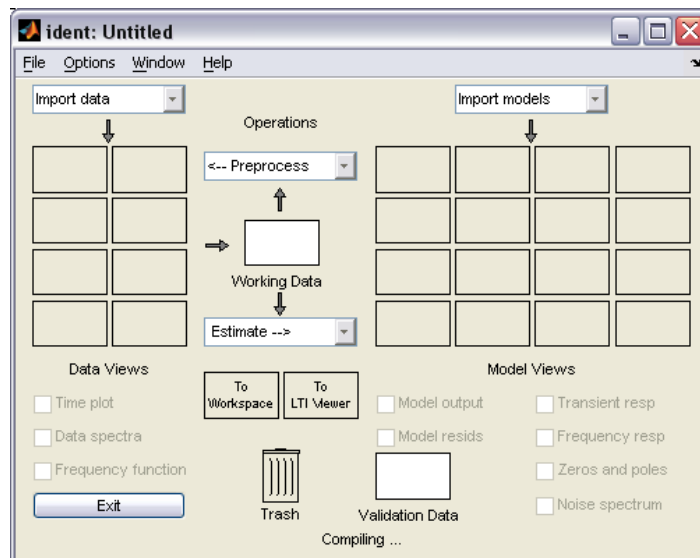


Figure 14 Matlab identification interface

- Select the Import Data drop-down tab then Time Domain Data. Fill in the dialogue as shown in Figure 15 then select Import.

Important. Remember to specify the proper sampling time. For this example a time of 0.001 [s] is set. For other Feedback control instrumentation the sampling time may be different. The value it is set to is mentioned in each of the Control Experiments manuals.

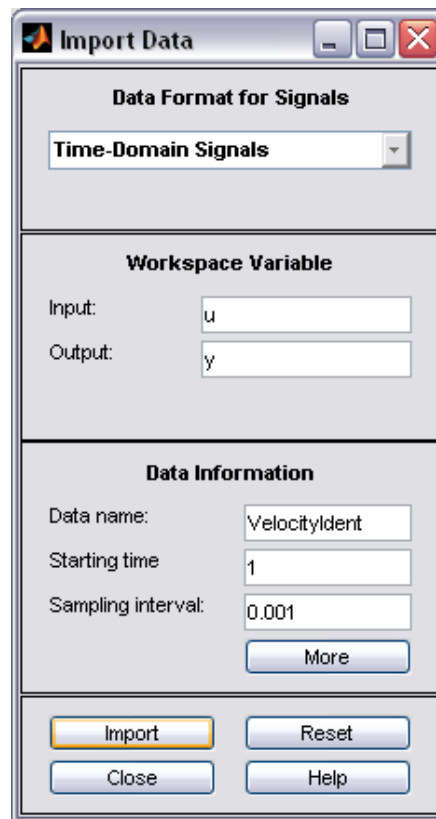


Figure 15 Import data dialogue

- The *ident* window will appear as shown in Figure 16.

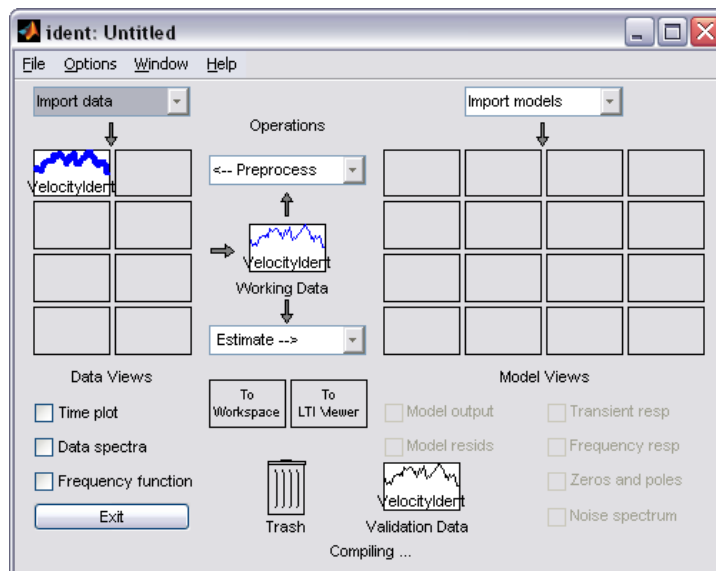


Figure 16 Data imported into ident window

- Now select *Estimate* → then *Parametric Model*. Select the *OE structure* and enter the order specified in the proper exercise of the Control Experiments manual (Figure 17). For the PMS velocity model it is the OE 1 1 1. Now select *Estimate*.

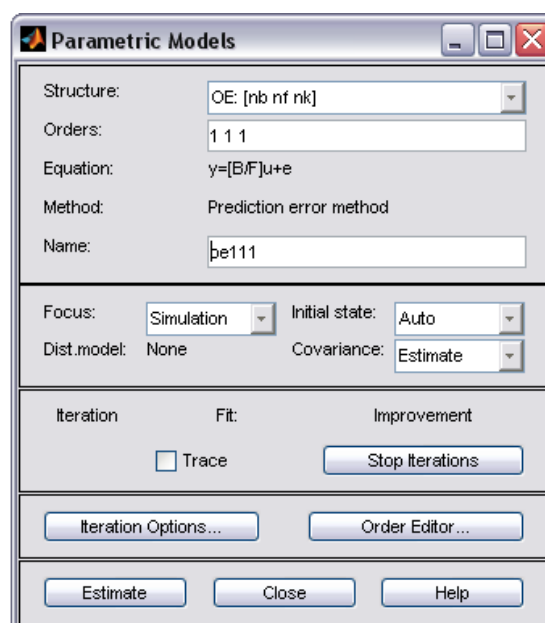


Figure 17 Parametric model dialogue

The estimated model will appear in the list of imported models (Figure 18).

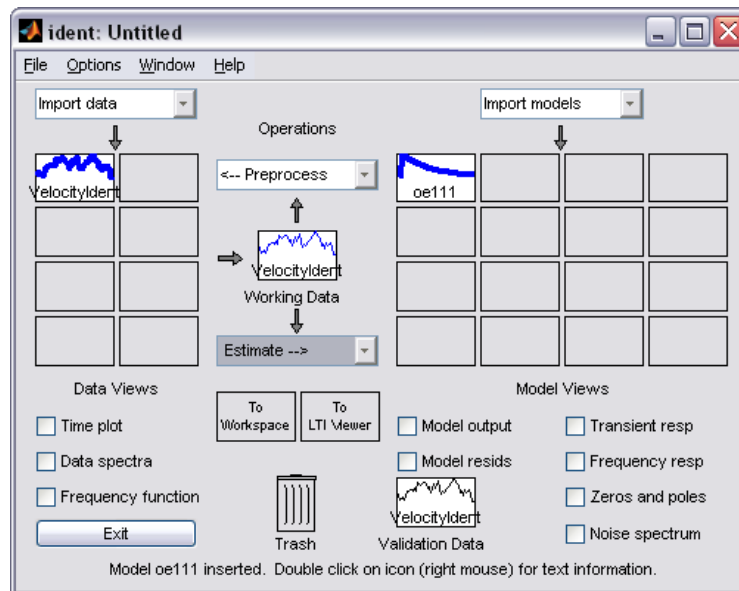


Figure 18 Model created and ready for analysis

- You can check the quality of the response of the identified model by the step response analysis, transient response, pole and zeros map, frequency response and model residuals. Select any of the model views to view the results, for example the transient response (Figure 19) or the model output (Figure 20).

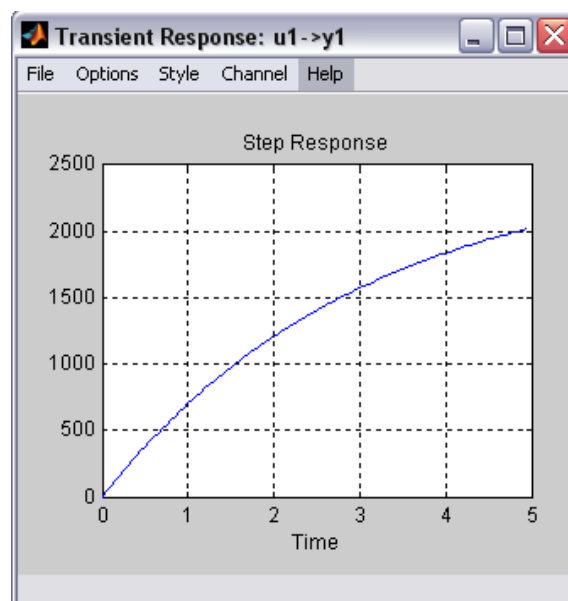


Figure 19 Step response

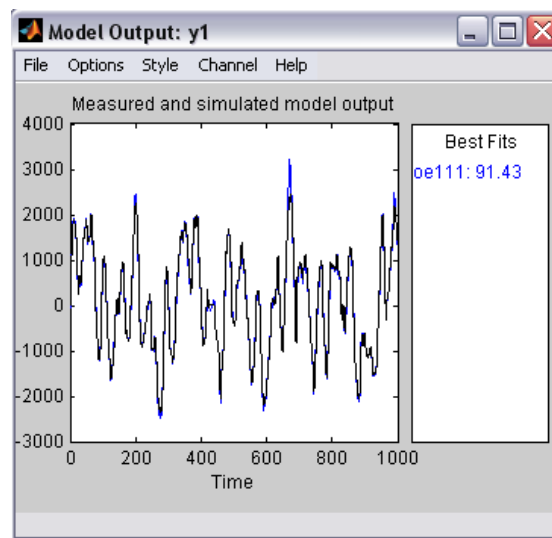


Figure 20 Model response vs. measured response

- Drag the oe111 (valid for PMS velocity model identification) model icon to the *To Workspace* box to export it to a workspace variable. Save the session using the *File – Save session as..* menu.
- The model is now available in Workspace, however it is available as a structure. For convenience the structure elements can be imported into a discrete and continuous transfer function. For discrete transfer function creation you may use one of the following commands:

```
>> sysd=filt(oe111.b, oe111.f, 0.001)
```

or

```
>> sysd=tf(oe111.b, oe111.f, 0.001)
```

Remember to specify the proper sampling time. For the PMS velocity model identification it is the 0.001 [s].

- For the PID controller design exercises the continuous models are used. In order to transform the discrete models into their continuous equivalents use the following command:

```
>> sysc=d2c(sysd, 'zoh')
```

PID controller design

The theoretical introduction to PID controller design is given in each of the Control Experiments manuals. The following exercise presents how PID controller design can be aided by MATLAB® 'rltool' function, which is toolbox for root locus design.

Exercise 6 – PID controller design

Introduction

This exercise serves as a guide for all of the PID controller design tasks in the Feedback Control Experiments manuals for Feedback Control Instrumentation.

Walkthrough

- Open root locus tool – 'rltool' command – the following window will open:

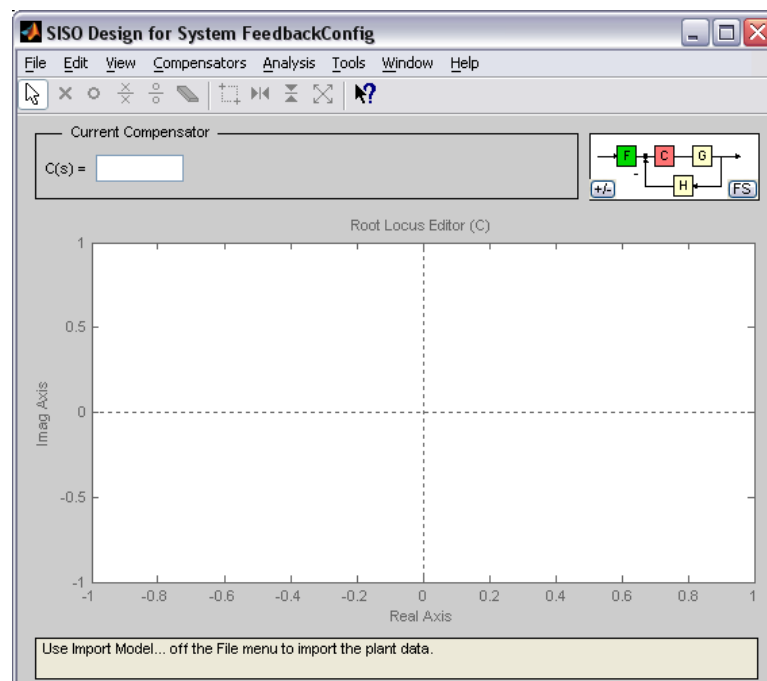


Figure 21 Root locus design window

- In this tool you can import a transfer function from the Workspace and assign it to plant $G(s)$, compensator/controller $C(s)$, $H(s)$ sensor and $F(s)$ pre-filter (*File*→*Import*). Import the transfer function 'sysc' of the plant from workspace into $G(s)$. Initially a root locus for closed loop

system is drawn for a proportional controller with gain = 1, thus $C(s) = 1$. To design a PID controller you can create a proper transfer function in the Workspace and import it into the root locus tool as $C(s)$. Then you can modify the system response by changing zeros, poles and the controller gain. Open the closed loop step response window (*Analysis-> Response to Step Command*). Observe how the closed loop system step response changes when you move the poles, zeros and change the gain of the controller.

- An example of a root locus plot of PMS velocity model with a PID controller is presented in Figure 22

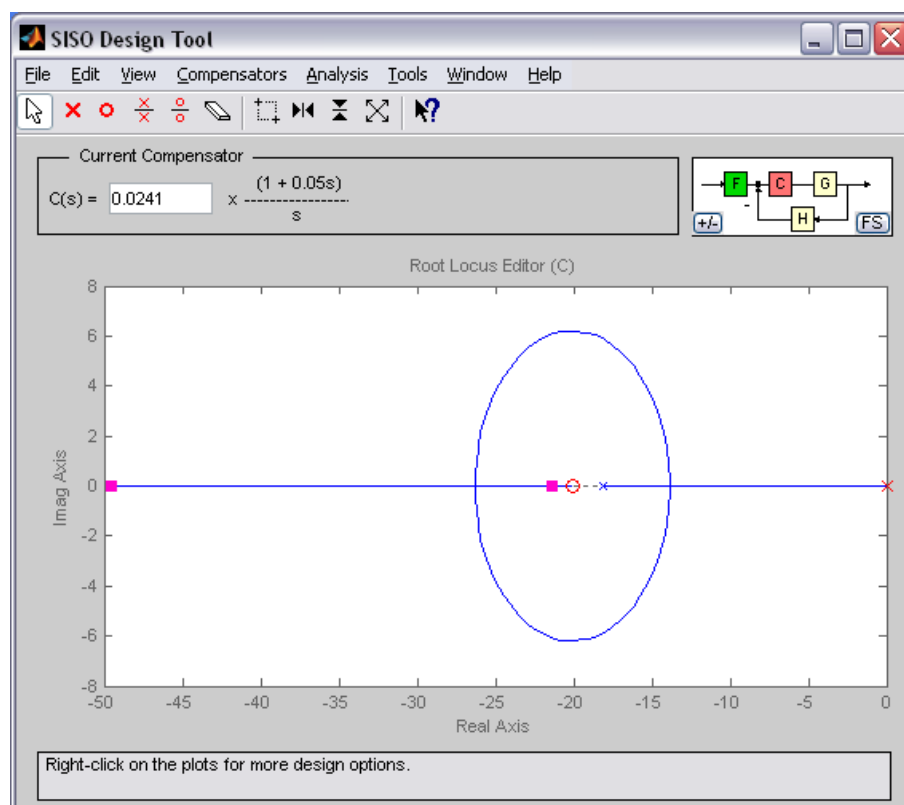


Figure 22 Root locus with PID controller

You can move the poles, zeros and the gain to obtain for example faster step response of the closed loop system. Then you can export the controller into Workspace (*File -> Export*) and test it on the model of the device. The tested controller can be then implemented in real-time control.